JavaScript

ECMAScript 2015 (6th Edition, ECMA-262)

Sources

- Mozilla Developer Network: JavaScript
  - https://developer.mozilla.org/en-US/docs/JavaScript

- ECMAScript Language Specification
  - Standard ECMA-262 / 6th Edition / June 2015
  - http://www.ecma-international.org/ecma-262/6.0/

const

- const name1 = value1 [, name2 = value2 [, ... [, nameN = valueN]]];
  - const a = {key: "value"}
- constants are block-scoped
  - like variables defined using the let statement
- the value of a constant cannot
  - change through reassignment
    - a = 10 -> Uncaught TypeError: Assignment to constant variable.
  - be redeclared (with let, const, var)
    - var a = 10 -> Uncaught SyntaxError: Identifier 'a' has already been declared
- a read-only reference to a value
  - the value it holds can be altered
    - a.key = 10 -> a = {key: 10}

arrow function expression

- (param1, …, paramN) => { statements }
- (param1, …, paramN) => expression
  - equivalent to: (param1, …, paramN) => { return expression; }
  - (param1, …, paramN) => ({foo: bar})
    - to return an object literal expression -> parenthesize the body of function
- param => { statements }
  - equivalent to: (param) => { statements }
  - function with one parameter -> parentheses are optional
- () => { statements }
  - function with no parameters
- a syntactically compact alternative to a regular function expression

arrow function expression

- no own bindings to **this**, **arguments**, **super**, or **new.target** keywords
  - the value of the enclosing **lexical scope** is used
  - arrow functions follow the normal variable lookup rules

```
function Person() {
  var that = this;
  that.age = 0;

  setInterval(function growUp() {
    that.age++;
  }, 1000);
}
```

```
function Person(){
  this.age = 0;


  setInterval(() => {
    this.age++;
  }, 1000);
}
```

arrow function and objects

- no own **this** => not well suited for object method functions

```
var obj = {
  i: 10,
  b: () => console.log(this.i, this),
  c: function() { console.log(this.i, this); }
}
obj.b(); // prints undefined, Window {...}
obj.c(); // prints 10, Object {i: 10, b: ƒ, c: ƒ}
```

- cannot be used as constructors

```
var Foo = () => {};
var foo = new Foo(); // TypeError: Foo is not a constructor
```

arrow function expression

- strict mode
  - **this** comes from the surrounding lexical context
    - strict mode rules with regard to this are ignored.
      - var f = () => { 'use strict'; return this; };
      - f() === window; // or the global object
    - all other strict mode rules apply normally
- **call()** or **apply()** can only pass in parameters, **thisArg** is ignored
- no binding of **arguments**, …

```
function foo(n) {
  var f = () => arguments[0] + n;
  return f(10); // == n + n
}
```

```
function foo(n) {
  var f = (...args) => args[0] + n;
  return f(10); // == 10 + n
}
```

The “class” syntax

prototype-based class User

```
function User(name) {
  this.name = name;
}

User.prototype.sayHi = function() {
  alert(this.name);
}

let user = new User("John");
user.sayHi();
```

“class” syntax-based class User

```
class User {
  constructor(name) {
    this.name = name;
  }
  sayHi() {
    alert(this.name);
  }
}

let user = new User("John");
user.sayHi();
```

Destructuring assignment

```
var a, b;
[a=5, b=7] = [1];
console.log(a); // 1
console.log(b); // 7
```

```
var a = 1;
var b = 3;
[a, b] = [b, a];
console.log(a); // 3
console.log(b); // 1
```

```
var o = {p: 42, q: true};
var {p, q} = o;
console.log(p); // 42
console.log(q); // true
```

```
var o = {p: 42, q: true};
var {p: foo, q: bar} = o;
console.log(foo); // 42
console.log(bar); // true
```

```
var {a = 10, b = 5} = {a: 3};
console.log(a); // 3
console.log(b); // 5
```

Spread syntax

```
var parts = ['shoulders', 'knees'];
var lyrics = ['head', ...parts, 'and', 'toes'];
// ["head", "shoulders", "knees", "and", "toes"]

function myFunction(v, w, x, y, z) { }
var args = [0, 1];
myFunction(-1, ...args, 2, ...[3]);
```

```
var obj1 = { foo: 'bar', x: 42 };
var obj2 = { foo: 'baz', y: 13 };

var clonedObj = { ...obj1 };
// Object { foo: "bar", x: 42 }

var mergedObj = { ...obj1, ...obj2 };
// Object { foo: "baz", x: 42, y: 13 }
```

Template literals (aka String interpolation)

- are enclosed by the back-tick (`)
- can contain placeholders indicated by the dollar sign and curly braces: **${expression}**

```
let name = `Richard`
let str = `Hello ${name}!`
```

Default function parameters

- function [name]([param1[= defaultValue1][, ..., paramN[= defaultValueN]]]) { statements }

```
function multiply(a, b = 1) {
  return a * b;
}

multiply(5, 2); // 10
multiply(5);    //  5
```

Rest parameters

- function's parameter can be prefixed with ...
  - which will cause all remaining (user supplied) arguments to be placed within a “standard” javascript array
  - **only the last** parameter can be a “rest parameter”

```
function myFun(a, b, ...manyMoreArgs) {
  console.log("a", a);
  console.log("b", b);
  console.log("manyMoreArgs", manyMoreArgs);
}

myFun("one", "two", "three", "four", "five");

// a, one
// b, two
// manyMoreArgs, [three, four, five]
```